WEEK 1 · LESSON 4 OF 24 · HANDOUT

Backpropagation: How Networks Learn

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=Ilg3gGewQ5U — open the lesson online to watch inside the course.

🎓 **Go deeper:** 3Blue1Brown: Backpropagation calculus — https://www.youtube.com/watch?v=tIeHLnjs5U8

🎓 **Go deeper:** Andrej Karpathy: Neural networks and backpropagation, step by step (building micrograd) — https://www.youtube.com/watch?v=VMj-3S1tku0

Your goal in this lesson: Explain backpropagation with the chain rule, and write a two-layer neural network in NumPy that learns the curved two-moons pattern.

The problem backpropagation solves

A network can have millions of weights. For each one we need its gradient: "if I change this weight a little, how does the loss change?" Nudging each weight one by one would take forever. **Backpropagation** computes **all** the gradients in one backward sweep through the network — about the same cost as one forward pass. It is the engine of all deep learning.

The chain rule in one picture

If the loss depends on the output, the output depends on the hidden layer, and the hidden layer depends on a weight, then:

> *how loss changes with the weight = (how loss changes with the output) × (how output changes with the hidden layer) × (how hidden layer changes with the weight)*

Backpropagation starts at the loss and walks **backwards**, multiplying these local slopes layer by layer and reusing them — that is why it is fast.

The network

- Input: 2 numbers (the moon coordinates).
- Hidden layer: 16 ReLU neurons.
- Output: 1 sigmoid neuron (probability of moon 1).
- Loss: binary cross-entropy.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

X, y = make_moons(n_samples=1000, noise=0.2, random_state=0)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
print(X_train.shape, X_test.shape)
plt.scatter(X[:, 0], X[:, 1], c=y, s=8)
plt.title("Two moons: a pattern a straight line cannot separate")
plt.show()
```

Forward and backward, step by step

```
rng = np.random.default_rng(0)
H = 16
W1 = rng.normal(0, 1, (2, H))          # input -> hidden
b1 = np.zeros(H)
W2 = rng.normal(0, 1, (H, 1)) * np.sqrt(1 / H)   # hidden -> output
b2 = np.zeros(1)

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def forward(X):
    z1 = X @ W1 + b1
    a1 = np.maximum(0, z1)              # ReLU
    z2 = a1 @ W2 + b2
    p = sigmoid(z2).ravel()
    return z1, a1, p

lr = 0.5
losses = []
for epoch in range(2000):
    z1, a1, p = forward(X_train)
    n = len(y_train)
    losses.append(-np.mean(y_train * np.log(p + 1e-9) + (1 - y_train) * np.log(1 - p + 1e-9)))

    # backward pass (chain rule, from the loss back to the first layer)
    dz2 = (p - y_train)[:, None] / n    # loss -> output score
    dW2 = a1.T @ dz2                    # -> weights of layer 2
    db2 = dz2.sum(axis=0)
    da1 = dz2 @ W2.T                    # -> hidden activations
    dz1 = da1 * (z1 > 0)                # -> through ReLU (slope 1 or 0)
    dW1 = X_train.T @ dz1               # -> weights of layer 1
    db1 = dz1.sum(axis=0)

    # gradient descent step
    W1 -= lr * dW1; b1 -= lr * db1
    W2 -= lr * dW2; b2 -= lr * db2

test_acc = ((forward(X_test)[2] > 0.5) == y_test).mean()
print("final training loss:", round(losses[-1], 3))
print("test accuracy:", test_acc)
```

Read the backward pass from top to bottom: each line is one link of the chain rule. The ReLU passes the gradient through where the neuron was active (z1 > 0) and blocks it where it was off.

The network reaches about **0.96** test accuracy, up from 0.86 with a single neuron. The hidden layer learned to bend the boundary.

See the curved boundary

```
xx, yy = np.meshgrid(np.linspace(-2, 3, 300), np.linspace(-1.5, 2, 300))
grid = np.c_[xx.ravel(), yy.ravel()]
zz = forward(grid)[2].reshape(xx.shape)
plt.contourf(xx, yy, zz > 0.5, alpha=0.3)
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, s=10)
plt.title("Decision boundary learned by a 16-neuron hidden layer")
plt.show()

plt.plot(losses)
plt.xlabel("epoch")
plt.ylabel("loss")
plt.show()
```

Why random starting weights?

If every weight started at zero, every hidden neuron would compute the same thing and receive the same gradient — they would stay identical forever. Random initial weights **break the symmetry** so each neuron can learn something different. Scaling them sensibly (here by $\sqrt{(1/H)}$) keeps the signals a healthy size (Lesson 10).

You will rarely write this again

From the next lesson, **PyTorch** computes the backward pass for you automatically. But now you know what `loss.backward()` really does: exactly the chain-rule steps above, for any network you build.

Key terms

- **Backpropagation** — computing every gradient in one backward sweep using the chain rule.
- **Chain rule** — multiply the slopes of each step to get the overall slope.
- **Hidden layer** — lets the network learn curved boundaries.
- **Symmetry breaking** — random initial weights so neurons learn different things.

Remember this: Backpropagation walks backwards from the loss, multiplying local slopes, to give every weight its gradient — and a single hidden layer already turns a straight line into a curve.

Name: _____ Date: _____

Activity

Task 1

Change the hidden layer to 2, 4 and 64 neurons. How does the test accuracy and the boundary change?

Task 2

Start all weights at zero. What happens to training, and why?

Task 3

Add a second hidden layer (16 → 16). Write its backward lines by following the same pattern.

Task 4

Use the gradient-check trick from Lesson 3 on one entry of W1 to confirm your backward pass.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does backpropagation compute?
   - A. The gradient of the loss for every weight
   - B. The predictions
   - C. The test accuracy
   - D. The number of layers
2. Which mathematical rule is backpropagation built on?
   - A. Bayes' theorem
   - B. The chain rule
   - C. Pythagoras' theorem
   - D. The central limit theorem
3. In which direction does backpropagation move through the network?
   - A. From input to output
   - B. From the loss back towards the input
   - C. Only through the first layer
   - D. Randomly
4. How does ReLU pass gradients backwards?
   - A. It turns them into probabilities
   - B. It doubles them
   - C. It passes them where the neuron was active and blocks them where it was off
   - D. It always blocks them
5. Why are initial weights random instead of all zero?
   - A. Random weights are always correct
   - B. PyTorch requires it
   - C. Zero weights use more memory
   - D. To break symmetry so neurons learn different things
6. Adding a 16-neuron hidden layer raised accuracy on the moons from about 0.86 to about 0.96. Why?
   - A. The test set got easier
   - B. The learning rate was removed
   - C. The hidden layer lets the boundary curve
   - D. The data changed

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.